1.

What is the purpose of a friend function in C++?

A. To access private members of a class
B. To access public members of a class
C. To inherit from a base class
D. To overload operators

Answer: A

2.

What is inheritance in C++?

A. A process of creating a new class from an existing class
B. A process of creating a new object from an existing object
C. A process of overloading operators
D. A process of declaring friend functions

Answer: A

3.

Which mode of inheritance makes all public members of the base class private in the derived class?

A. Public inheritance
B. Private inheritance
C. Protected inheritance
D. Multiple inheritance

Answer: B

4.

What type of mode of inheritance is used in the following code snippet?

```
#include<iostream>
class Base
{
	public:
		void print()
		{
			std::cout << "Base class" << std::endl;
		}
};
class Derived : Base
{
	public:
		void printDerived()
		{
			std::cout << "Derived class" << std::endl;
		}
};
```

A. Private mode of inheritance
B. Public mode of inheritance
C. Protected mode of inheritance
D. Multiple mode of inheritance

Answer: A

5.

What will be the access specifier of the members of the Base class in the Derived class ?

```
#include<iostream>
class Parent {
	public:
		void printPublic() {
			std::cout << "Public member" << std::endl;
		}
	protected:
		void printProtected() {
			std::cout << "Protected member" << std::endl;
		}
	private:
		void printPrivate() {
			std::cout << "Private member" << std::endl;
		}
};
class Child : private Parent {
};
```

A. Public members will remain public, protected members will remain protected, and private members will remain private
B. Public members will become private, protected members will become private, and private members will rename private.
C. Public members will become protected, protected members will remain protected, and private members will remain private
D. Public members will remain public, protected members will remain protected, and private members will become public

Answer: B

6.

Which of the following operators can be overloaded in C++ ?

A. sizeof

B. dot operator .

C. scope resolution operator ::

D. plus operator +

Answer: D